1. Container và góc nhìn

A. 4 Máy tính ảo chạy ngầm (4 Containers):

1. **Máy Người dùng (Client - IP: 172.20.0.10):** Giả lập một người dùng bình thường đang sử dụng ứng dụng.
2. **Máy Chủ (Target Server - IP: 172.20.0.20):** Nơi chứa trang web (App), cơ sở dữ liệu (tài khoản, mật khẩu, tài nguyên) để phục vụ Client.
3. **Máy Hacker (Hacker Node - IP: 172.20.0.66):** Kẻ xấu nằm chung mạng, chứa các công cụ để nghe lén, phá hoại Client và Server.
4. **Máy Giám sát (God-View - IP: 172.20.0.99):** "Đài quan sát Thượng đế" — đứng ngoài cuộc chiến, thu thập thông tin từ 3 máy kia để vẽ lên màn hình.

B. 2 Màn hình hiển thị (Khi bạn chạy Demo):

- **Màn hình 1 — Giao diện App cơ bản (Góc nhìn Người dùng):**
  - Rất đơn giản, chỉ gồm: **Trang Đăng nhập** (nhập Username/Password) và **Trang Tài nguyên** (sau khi đăng nhập thành công thì hiện thông tin cá nhân, danh sách file hoặc giữ một kết nối tải dữ liệu).
- **Màn hình 2 — Giao diện "Góc nhìn Thượng đế" (God-View Dashboard - Trọng tâm của bạn):**
  - Màn hình này chia làm các ô:
    - **Sơ đồ mạng ở giữa:** Vẽ 3 cục Client, Server, Hacker và các chấm màu chạy qua lại biểu thị gói tin đang bay.
    - **Các bảng thông số bên cạnh:** Bảng địa chỉ mạng (ARP Table), Bảng kết nối (TCP Table), và Cửa sổ dòng lệnh (Terminal Log) báo cáo từng giây chuyện gì đang xảy ra.

2. Chúng kết nối với nhau như thế nào? (Hai luồng mạng)

Để không bị rối, bạn chỉ cần nhớ hệ thống có **2 luồng giao tiếp** tách biệt:

1. **Luồng 1 — Luồng Đời thực (Giữa Client, Server và Hacker):**
   - Client gửi yêu cầu đăng nhập hoặc lấy dữ liệu tới Server (qua giao thức HTTP/TCP).
   - Hacker nhảy vào giữa luồng này để nghe lén, bơm gói tin giả, hoặc bắn phá Server.
2. **Luồng 2 — Luồng Báo cáo (Gửi về cho God-View):**

- Làm sao màn hình Thượng đế biết chuyện gì đang xảy ra? Đó là nhờ trên cả 3 máy (Client, Server, Hacker) đều có gắn một đoạn code nhỏ (gọi là **Agent/Logger**).
- Cứ mỗi giây, hoặc mỗi khi có sự kiện (ví dụ: Client bấm nút Login, Hacker bắt được mật khẩu, Server thấy mình bị quá tải), đoạn code đó sẽ gửi một bản tin **JSON** về cho God-View (172.20.0.99).
- God-View nhận được bản tin JSON đó liền đẩy ngay lập tức lên trình duyệt bằng **WebSocket** để màn hình của bạn đổi màu, nhảy số theo thời gian thực (Real-time).

3. Có mấy cách tấn công và mấy loại tấn công trong đề tài?

Theo đúng đề bài bạn nêu (*"tấn công trên đường truyền hoặc tấn công thẳng vào app/server"*) và khớp với bảng phân công của nhóm bạn, các đòn tấn công được chia theo **2 hướng tiếp cận** và **3 loại kỹ thuật cụ thể** (ở giai đoạn đầu):

Phân loại theo HƯỚNG TIẾP CẬN (2 cách):

1. **Cách 1: Tấn công trên đường truyền (Giữa Client và Server):** Server không hề bị lỗi code, nhưng khi gói tin rời khỏi máy Client đi trên dây mạng để tới Server, Hacker đứng ở giữa chặn lại để đọc trộm hoặc cắt đứt dây.
2. **Cách 2: Tấn công thẳng vào Server / App:** Hacker không thèm quan tâm Client là ai, mà nhắm thẳng vào địa chỉ IP của Server (172.20.0.20) để bắn phá làm sập máy chủ, hoặc gửi dữ liệu độc hại vào form đăng nhập để lừa cơ sở dữ liệu.

Phân loại theo KỸ THUẬT CỤ THỂ:

Tên đòn tấn công	**Thuộc hướng nào?**	**Chuyện gì hiện lên màn hình God-View?**	**Cách phòng thủ (Trung cấu hình trên Server)**
1. ARP Spoofing + Sniffing *(Giả mạo địa chỉ & Nghe lén)*	**Trên đường truyền**	• Đường truyền thay vì đi thẳng Client -> Server thì bị bẻ cong đi vòng qua Hacker. • **Bảng ARP** đổi sang màu đỏ	Thiết lập bảng ARP tĩnh (Static ARP) hoặc mã hóa đường truyền (HTTPS) để Hacker có bắt được cũng không đọc được chữ.

		vì địa chỉ máy bị làm giả. • Khung Log hiện rõ mật khẩu bị lộ.	
2. TCP SYN Flood *(Làm ngập lụt kết nối)*	**Thẳng vào Server**	• Hàng trăm chấm đỏ bay dồn dập từ Hacker vào Server. • **Bảng TCP** của Server báo số lượng chờ (SYN_RECV) tăng vọt lên cực đại. • Bên màn hình App của Client báo lỗi không vào được web.	Bật tính năng **syn_cookies=1** trong hệ điều hành Linux của Server (giúp Server không bị giữ chỗ ảo nữa).
3. TCP RST Injection *(Giả mạo lệnh Cắt đứt kết nối)*	**Trên đường truyền + Vào kết nối**	• Một tia màu đỏ từ Hacker bắn thẳng vào sợi dây kết nối giữa Client và Server. • Dòng trạng thái ESTABLISHED (Đang kết nối) trong **Bảng TCP** biến mất ngay lập tức.	Chặn việc nghe lén từ bước 1 (vì Hacker phải nhìn trộm được mã số thứ tự gói tin trên đường truyền mới giả mạo lệnh RST khớp được).

*(Sang giai đoạn sau, nếu nhóm muốn mở rộng thêm phần "Tấn công thẳng vào App", nhóm có thể thêm đòn thứ 4 rất kinh điển là **SQL Injection**: Hacker nhập câu lệnh ' OR 1=1 -- vào ô đăng nhập để vào thẳng tài khoản Admin mà không cần biết mật khẩu).*

4. Tóm tắt một kịch bản chạy Demo hoàn chỉnh (Từ A đến Z)

1. **Bước 1 (Bình thường):** Bạn mở màn hình **God-View** lên. Sơ đồ mạng màu xanh bình yên. Bạn mở tab **App**, gõ tài khoản/mật khẩu bấm Đăng nhập → Trên màn hình God-View có 1 hạt màu xanh bay từ Client sang Server, bảng TCP hiện ESTABLISHED (kết nối thành công).
2. **Bước 2 (Tấn công đường truyền):** Trí kích hoạt đòn **ARP Spoofing** → Ngay lập tức trên màn hình God-View của bạn, Bảng ARP nháy đỏ báo động, đường dây nối tự động bẻ cong qua nút Hacker. Khi người dùng đăng nhập lại, trên màn hình God-View hiện lên dòng chữ màu đỏ ở góc Hacker: *"Đã bắt được mật khẩu: 123456"*.
3. **Bước 3 (Tấn công Server):** Trí kích hoạt đòn **SYN Flood** → Màn hình God-View hiện cơn mưa chấm đỏ bắn vào Server, cột biểu đồ quá tải của Server dâng lên mức 100%. Bạn quay sang tab **App** bấm tải lại trang → Trang web bị đơ (chứng minh Server đã tê liệt).
4. **Bước 4 (Phòng thủ):** Trung bật công tắc bảo vệ (syn_cookies = 1) trên Server → Màn hình God-View hiện biểu tượng cái khiên trên Server, số kết nối ảo giảm xuống, và người dùng bên tab **App** lại truy cập được bình thường.

TÀI LIỆU THIẾT KẾ KIẾN TRÚC HỆ THỐNG & GIAO THỨC TRUYỀN TIN

Mã hạng mục: T1-ALL (Milestone 1 - Tuần 1)

Thành viên thực hiện: Trung (T1-TV1), Trí (T1-TV2), Nguyên (T1-TV3)

1. Kiến trúc Mạng Docker (Docker Network Topology)

1.1. Quy hoạch Subnet & Địa chỉ IP tĩnh (Static IP)

Toàn bộ 4 container nằm trong một mạng Bridge tự định nghĩa tên là cyber_net với dải mạng 172.20.0.0/24 (Gateway: 172.20.0.1).

Tên Container	Vai trò	Địa chỉ IP tĩnh	Port nội bộ	Port máy Host	Phụ trách chính
client_node	Giả lập người dùng (Gửi HTTP Login & giữ kết nối TCP)	172.20.0.10	3000	3000	Trung (T1-TV1)
target_server	App Server + Database (SQLite) + OS Agent đọc bảng mạng	172.20.0.20	8080 (HTTP) 9000 (TCP)	8080	Trung (T1-TV1)

hacker_node	Máy tấn công (Chứa script ARP, SYN Flood, TCP RST)	172.20.0.66	5000 (Control API)	*Không mở*	Trí (T1-TV2)
god_view	WebSocket Server thu thập log & Giao diện Thượng đế	172.20.0.99	4000 (WS/API) 5173 (Web UI)	4000 5173	Nguyên (T1-TV3)

Lưu ý chống nghẽn Telemetry (Quan trọng): Khi Trí chạy **TCP SYN Flood** bằng hping3, chỉ bắn gói tin từ 172.20.0.66 vào thẳng 172.20.0.20:8080 (hoặc :9000) với tốc độ vừa phải (--fast hoặc --interval u1000, tránh dùng --flood tối đa làm treo Docker daemon của máy Host) để các bản tin log gửi về 172.20.0.99:4000 không bị rớt.

2. Cấu hình Hệ điều hành & Docker Compose (T1-TV1 - Trung)

Để phục vụ 3 đòn tấn công L2/L4 của Trí (ARP Spoofing, SYN Flood, TCP RST Injection), file docker-compose.yml do **Trung** viết cần đảm bảo các thông số kỹ thuật sau:

2.1. Quyền hạn Linux Capabilities & Sysctls

- **Trên target_server (172.20.0.20)**:
  - Cấp quyền cap_add: [NET_ADMIN] để có thể can thiệp cấu hình mạng lúc chạy demo.
  - Cấu hình sysctls ban đầu (Chế độ dễ bị tổn thương - Vulnerable Mode):
    - net.ipv4.tcp_syncookies=0 (Tắt chống SYN Flood để hàng đợi SYN_RECV bị đầy khi Trí tấn công).
    - net.ipv4.tcp_max_syn_backlog=128 (Hạ thấp hàng đợi backlog để hiệu ứng đầy hàng đợi diễn ra nhanh, dễ nhìn trên God-View).

- **Trên hacker_node (172.20.0.66)**:
    - Cấp quyền cap_add: [NET_ADMIN, NET_RAW] (Bắt buộc để chạy arpspoof, scapy, hping3, tcpkill).
    - Cấu hình sysctls: net.ipv4.ip_forward=1 (Cho phép gói tin đi xuyên qua máy Hacker khi thực hiện ARP Spoofing mà không làm đứt mạng của Client).

2.2. Khung docker-compose.yml chuẩn (Dành cho Trung khởi tạo)

YAML

```
version: '3.8'

services:

  client_node:

    build: ./client

    container_name: client_node

    cap_add:

      - NET_ADMIN

    networks:

      cyber_net:

        ipv4_address: 172.20.0.10



  target_server:

    build: ./server

    container_name: target_server

    ports:

      - "8080:8080"

    cap_add:
```

```
    - NET_ADMIN
  sysctls:
    - net.ipv4.tcp_syncookies=0
    - net.ipv4.ip_forward=0
  networks:
    cyber_net:
      ipv4_address: 172.20.0.20

hacker_node:
  build: ./hacker
  container_name: hacker_node
  cap_add:
    - NET_ADMIN
    - NET_RAW
  sysctls:
    - net.ipv4.ip_forward=1
  networks:
    cyber_net:
      ipv4_address: 172.20.0.66

god_view:
  build: ./god_view
  container_name: god_view
```

```
    ports:
      - "4000:4000"
      - "5173:5173"
    networks:
      cyber_net:
        ipv4_address: 172.20.0.99


networks:
  cyber_net:
    driver: bridge
    ipam:
      config:
        - subnet: 172.20.0.0/24
          gateway: 172.20.0.1
```

2.3. Các gói công cụ mạng cần cài sẵn trong Dockerfile

Để tránh lỗi thiếu lệnh khi chạy thực tế, các container sử dụng base python:3.11-slim hoặc ubuntu:22.04 và cài đặt:

- **client_node & target_server**: iproute2 (lệnh ip neigh, ss), net-tools (arp, netstat), procps (sysctl), curl.
- **hacker_node**: iproute2, net-tools, dsniff (chứa arpspoof và tcpkill), hping3, tcpdump, thư viện Python scapy và requests.

3. Đặc tả Kỹ thuật 3 Kịch bản Tấn công Mạng (T1-TV2 - Trí)

Để giao diện của **Nguyên (T1-TV3)** hiển thị chính xác bảng **ARP**, bảng **TCP** và **Terminal Log**, cơ chế hoạt động của 3 đòn tấn công trong Tuần 1 được thống nhất như sau:

Kịch bản 1: ARP Spoofing + Sniffing (Tấn công Lớp 2 - Data Link)

1. **Bình thường:** Bảng ARP (ip neigh / arp -n) của Client (172.20.0.10) ghi nhận IP 172.20.0.20 ứng với địa chỉ MAC thật của Server (MAC_SERVER).
2. **Tấn công:** Trí chạy arpspoof -i eth0 -t 172.20.0.10 -r 172.20.0.20.
3. **Hiện tượng cần lấy dữ liệu:**
    - Bảng ARP của Client và Server bị ghi đè: cả hai đều trỏ về địa chỉ MAC của Hacker (MAC_HACKER).
    - Khi Client gửi POST /api/login (chứa username & password), gói tin đi qua 172.20.0.66. Script Scapy của Trí bắt được gói HTTP này và gửi nội dung giải mã về cho God-View.

Kịch bản 2: TCP SYN Flood bằng hping3 (Tấn công Lớp 4 - Transport)

1. **Bình thường:** Bảng kết nối TCP trên Server (ss -tan) chỉ có vài kết nối ở trạng thái ESTAB hoặc LISTEN.
2. **Tấn công:** Trí chạy hping3 -S -p 8080 --rand-source --interval u1000 172.20.0.20 (Gửi hàng loạt cờ SYN từ IP giả mạo).
3. **Hiện tượng cần lấy dữ liệu:**
    - Trên Server, số lượng kết nối ở trạng thái SYN-RECV tăng vọt và lấp đầy hàng đợi.
    - Client thử gửi request đăng nhập nhưng bị **Timeout** (không thể bắt tay 3 bước với Server).
    - Khi Trung bật phòng thủ (sysctl -w net.ipv4.tcp_syncookies=1), Server kích hoạt SYN Cookies và Client đăng nhập lại được bình thường.

Kịch bản 3: TCP RST Injection bằng tcpkill / Scapy (Tấn công Lớp 4 - Cắt đứt kết nối)

1. **Bình thường:** Giữa Client (172.20.0.10) và Server (172.20.0.20) có một phiên kết nối TCP đang duy trì (ESTABLISHED - ví dụ phiên tải dữ liệu hoặc giữ kết nối).
2. **Tấn công:** Kết hợp khi đang ở giữa luồng (hoặc nghe lén Sequence Number), Trí dùng tcpkill -i eth0 host 172.20.0.10 and port 8080 hoặc script Scapy đọc seq/ack

của gói tin đang chạy rồi bơm một gói TCP giả mạo có cờ **RST (Reset)** vào Server và Client.

3. **Hiện tượng cần lấy dữ liệu:**
    - Kết nối ESTABLISHED trên bảng TCP của Server biến mất ngay lập tức.
    - Client nhận lỗi Connection reset by peer (Mất kết nối đột ngột).

4. Chuẩn hóa JSON Event Schema (Hợp đồng Dữ liệu cho T1-TV3 - Nguyên)

Tất cả các sự kiện từ Client, Server, Hacker đều được đẩy về God-View Server thông qua giao thức:

- **Endpoint nhận sự kiện (từ Trung & Trí gửi về):** POST http://172.20.0.99:4000/api/telemetry

- **Kênh đẩy lên UI (Nguyên phát lên trình duyệt):** WebSocket ws://localhost:4000 (event: "GOD_VIEW_UPDATE")

4.1. Cấu trúc JSON chuẩn (Envelope chung)

Mọi bản tin gửi về 172.20.0.99:4000/api/telemetry **bắt buộc** tuân theo khung JSON dưới đây:

JSON

```
{
  "timestamp": 1761143700000,
  "source_node": "CLIENT | SERVER | HACKER",
  "source_ip": "172.20.0.20",
  "event_type": "STATE_SNAPSHOT | PACKET_EVENT | ATTACK_STATUS | APP_EVENT",
  "severity": "NORMAL | WARNING | DANGER",
  "payload": {}
}
```

4.2. Chi tiết payload cho từng loại event_type (Cần khớp với UI của Nguyên)

Loại 1: STATE_SNAPSHOT (Cập nhật Bảng ARP & Bảng TCP mỗi 1-2 giây)

Do client_node và target_server của **Trung** định kỳ đọc từ hệ điều hành (ip neigh, ss -tan, sysctl) và gửi về để **Nguyên** vẽ **Bảng ARP/TCP** trên giao diện:

JSON


```
{
  "timestamp": 1761143701000,
  "source_node": "SERVER",
  "source_ip": "172.20.0.20",
  "event_type": "STATE_SNAPSHOT",
  "severity": "NORMAL",
  "payload": {
    "os_config": {
      "syn_cookies": 0,
      "ip_forward": 0
    },
    "arp_table": [
      { "ip": "172.20.0.10", "mac": "02:42:ac:14:00:0a", "is_spoofed": false },
      { "ip": "172.20.0.66", "mac": "02:42:ac:14:00:42", "is_spoofed": false }
    ],
    "tcp_summary": {
      "established_count": 1,
      "syn_recv_count": 0,
```

```
      "time_wait_count": 2
    },
    "tcp_connections": [
      {
        "local": "172.20.0.20:8080",
        "remote": "172.20.0.10:54322",
        "state": "ESTABLISHED"
      }
    ]
  }
}
```

(Mẹo hiển thị cho Nguyên: Khi Trí bật ARP Spoof, địa chỉ mac của 172.20.0.10 và 172.20.0.66 sẽ giống hệt nhau -> UI tự động tô đỏ dòng đó trong bảng ARP!)

Loại 2: PACKET_EVENT & ATTACK_STATUS (Dữ liệu từ Hacker của Trí gửi về)

Do script của **Trí** gửi về mỗi khi bắt đầu/kết thúc một đòn tấn công, hoặc khi bắt được gói tin nhạy cảm hay tiêm gói RST thành công:

JSON

```
{
  "timestamp": 1761143705000,
  "source_node": "HACKER",
  "source_ip": "172.20.0.66",
  "event_type": "PACKET_EVENT",
  "severity": "DANGER",
```

```
  "payload": {
    "attack_mode": "ARP_SPOOF | SYN_FLOOD | TCP_RST",
    "action": "INTERCEPTED | INJECTED | FLOODING",
    "packet_info": {
      "protocol": "HTTP | TCP | ARP",
      "src": "172.20.0.10:54322",
      "dst": "172.20.0.20:8080",
      "flags": "PA | S | R",
      "seq_num": 19482731,
      "extracted_data": "POST /api/login -> user=hanhnguyen&pass=secret123"
    },
    "terminal_log": "[ARP-SPOOF] Sniffed credentials from 172.20.0.10 ->
user=hanhnguyen"
  }
}
```

Loại 3: APP_EVENT (Sự kiện ứng dụng Đăng nhập / Truy cập tài nguyên)

Do client_node và target_server gửi về khi người dùng thao tác trên App:

JSON

```
{
  "timestamp": 1761143706000,
  "source_node": "CLIENT",
  "source_ip": "172.20.0.10",
  "event_type": "APP_EVENT",
```

```
  "severity": "NORMAL",

  "payload": {

    "action": "LOGIN_ATTEMPT | RESOURCE_FETCH | CONNECTION_RESET |
REQUEST_TIMEOUT",

    "target_ip": "172.20.0.20",

    "http_status": 200,

    "terminal_log": "[CLIENT] Gửi yêu cầu đăng nhập tới 172.20.0.20:8080..."

  }

}
```

5. Quy tắc Thiết kế Giao diện Mockup God-View (T1-TV3 - Nguyên)

Dựa trên JSON Schema ở Mục 4 và yêu cầu của task T1-TV3, màn hình God-View của **Nguyên** sẽ chia thành **4 khu vực trực quan**:

1. **Khu vực 1 - Sơ đồ Topology Mạng (Giữa màn hình)**:
    - Hiển thị 3 Node chính: Client (172.20.0.10), Server (172.20.0.20), Hacker (172.20.0.66).
    - **Trạng thái Bình thường:** Đường nối xanh lá thẳng từ Client ↔ Server.
    - **Khi attack_mode == "ARP_SPOOF":** Đường nối thẳng bị mờ đi, xuất hiện đường nối màu cam đi vòng: Client → Hacker → Server.
    - **Khi attack_mode == "SYN_FLOOD":** Hàng loạt chấm đỏ bắn liên tục từ Hacker → Server, Node Server nhấp nháy cảnh báo quá tải.
    - **Khi attack_mode == "TCP_RST":** Tia sét màu đỏ từ Hacker đánh vào đường nối Client - Server, làm đứt đôi đường kết nối.
2. **Khu vực 2 - Bảng ARP Cache (Góc trái dưới)**:
    - Hiển thị bảng ánh xạ IP -> MAC của Client và Server. Khi bị ARP Spoofing, dòng bị giả mạo chuyển sang nền đỏ (POISONED!).
3. **Khu vực 3 - Bảng Trạng thái TCP & OS Config (Góc phải trên)**:

    - Hiển thị thanh trạng thái syn_cookies: OFF/ON, bộ đếm SYN_RECV (tăng vọt khi bị SYN Flood) và danh sách các kết nối ESTABLISHED (biến mất ngay khi bị TCP RST Injection).
4. **Khu vực 4 - Live Terminal Log (Góc dưới cùng)**:
    - Nối chuỗi trường payload.terminal_log từ cả 3 node theo thứ tự thời gian, phân biệt màu chữ: Xanh dương (CLIENT), Xanh lá (SERVER), Đỏ (HACKER).

6. Checklist Nghiệm thu Cuối Tuần 1 (Milestone 1)

Để kết thúc Tuần 1 thành công và sẵn sàng tích hợp ở Tuần 2, mỗi thành viên kiểm tra đầu ra của mình theo tiêu chí sau:

- [ ] **Trung (T1-TV1)**: Chạy lệnh docker compose up -d lên đủ 4 container đúng IP tĩnh. Từ container client_node ping và curl http://172.20.0.20:8080 thành công. Chạy thử lệnh bật/tắt sysctl net.ipv4.tcp_syncookies trên target_server không báo lỗi.
- [ ] **Trí (T1-TV2)**: Mở terminal vào container hacker_node (docker exec -it hacker_node bash), chạy thử 3 lệnh/script (arpspoof, hping3, tcpkill/Scapy) tác động thực tế lên client_node và target_server.
- [ ] **Nguyên (T1-TV3)**: Server WebSocket tại 172.20.0.99:4000 nhận được bản tin JSON mẫu (test bằng Postman/curl) và đẩy thẳng lên giao diện Mockup UI làm thay đổi trạng thái sơ đồ mạng, bảng ARP/TCP và Terminal Log.

Quy trình Nối tiếp (Pipeline) - Kế hoạch Tuần 1

1.Chặng 0: Cả nhóm thống nhất quy ước

Phân hệ	**Người phụ trách**	**Địa chỉ / Cổng (Port)**	**Các Endpoint bắt buộc phải mở**
Target App & Server *(Client + Backend)*	Thành viên 1	http://localhost:3000	• POST /api/login: Nhận { username, password } trả về Token/Session. • GET /api/resources: Trả về danh sách tài nguyên mật (khi đã đăng nhập).
Attacker Controller *(Trạm điều khiển Hacker)*	Thành viên 2	http://localhost:5000 *(hoặc chạy qua CLI)*	• POST /attack/mitm: Bật/tắt chế độ nghe lén trên đường truyền. • POST /attack/sqli: Kích hoạt bắn payload SQL Injection vào cổng 3000. • POST /attack/dos: Kích hoạt bắn 50 requests/giây vào cổng 3000.
God-View Server & UI *(Trạm quan sát Thượng Đế)*	Thành viên 3	http://localhost:4000	• POST /api/telemetry: Nhận toàn bộ bản tin JSON từ TV1 và TV2 bắn về. • ws://localhost:4000: Luồng WebSocket đẩy dữ liệu thẳng lên giao diện Web.

CẤU TRÚC JSON

```
{
  "event_id": "evt_001",
  "timestamp": "2026-10-23T08:30:00.000Z",
```

```
  "scenario": "NORMAL_LOGIN | MITM_SNIFF | SQL_INJECTION | DOS_FLOOD",
  "actor": "CLIENT | NETWORK | SERVER | HACKER",
  "direction": "CLIENT_TO_SERVER | SERVER_TO_CLIENT | NETWORK_TO_HACKER | HACKER_TO_SERVER",
  "status": "INFO | COMPROMISED | DANGER | BLOCKED",
  "summary": "Mô tả ngắn gọn 1 dòng để hiện lên thanh trạng thái trên UI",
  "technical_details": {
    "http_method": "POST",
    "endpoint": "/api/login",
    "raw_payload": "Nội dung gói tin hoặc dữ liệu truyền đi",
    "server_Internal_execution": "Câu lệnh SQL thực tế hoặc trạng thái xử lý bên trong Server"
  }
}
```

Quy ước ý nghĩa các giá trị (Để Thành viên 3 lập trình hiệu ứng hình ảnh):

- **`direction` (Hướng bay của hạt dữ liệu trên sơ đồ):**
  - `CLIENT_TO_SERVER`: Vẽ hạt dữ liệu chạy từ cục **Client** sang cục **Server**.
  - `NETWORK_TO_HACKER`: Vẽ hạt dữ liệu rẽ nhánh từ **giữa đường truyền** rơi xuống cục **Hacker** (biểu diễn gói tin bị đánh cắp).
  - `HACKER_TO_SERVER`: Vẽ hạt dữ liệu màu đỏ bắn thẳng từ cục **Hacker** vào cục **Server**.
- **`status` (Quyết định màu sắc trên màn hình Thượng Đế):**
  - `INFO` (Màu Xanh dương/Xanh lá): Giao tiếp bình thường của người dùng.
  - `COMPROMISED` (Màu Vàng cam): Dữ liệu đã bị lộ hoặc bị chỉnh sửa trên đường truyền.
  - `DANGER` (Màu Đỏ nhấp nháy): Hệ thống đang bị tấn công trực diện hoặc bị vượt quyền thành công.

LOG MẪU

// Hàm chuẩn dùng chung cho Thành viên 1 và Thành viên 2

```
async function reportToGodView(actor, direction, status, summary, details = {}) {
  const payload = {
    event_id: "evt_" + Date.now(),
    timestamp: new Date().toISOString(),
    actor: actor,           // "CLIENT" | "SERVER" | "HACKER"
```

```
    direction: direction,   // "CLIENT_TO_SERVER" | "NETWORK_TO_HACKER" |
"HACKER_TO_SERVER"
    status: status,         // "INFO" | "COMPROMISED" | "DANGER"
    summary: summary,
    technical_details: details
  };

  try {
    await fetch("http://localhost:4000/api/telemetry", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(payload)
    });
  } catch (err) {
    console.error("Chưa bật God-View Server ở cổng 4000:", err.message);
  }
}
```

2.Chặng 1: Thành viên 1 xây dựng Môi trường Đích - Target App & Server:Đầu vào: Bản đặc tả yêu cầu | Đầu ra: App chạy được + API có lỗ hổng.

Thành viên 1 đóng vai trò là người đặt nền móng. Nếu chưa có App và Server thì Hacker không có gì để phá và Thượng Đế không có gì để xem.

- **Bước 1.1:** Khởi tạo Database cơ bản (bảng Users và bảng Resources chứa tài nguyên mật) cùng Backend Server.
- **Bước 1.2:** Xây dựng giao diện Client đơn giản (Form Đăng nhập & Trang hiển thị tài nguyên sau khi đăng nhập thành công).
- **Bước 1.3:** Cố tình để lộ 2 điểm yếu kỹ thuật theo đúng đề tài:
    1. *Trên đường truyền:* Cấu hình gửi thông tin đăng nhập qua giao thức không mã hóa (hoặc qua 1 cổng Proxy trung gian mở sẵn) để lộ thông tin nhạy cảm.
    2. *Trên Server:* Viết câu truy vấn kiểm tra đăng nhập dạng cộng chuỗi trực tiếp (tạo lỗ hổng SQL Injection) và không giới hạn số lần thử sai (tạo lỗ hổng Brute-force).
-
- **Bước 1.4:** Viết bộ phát tín hiệu (Logger Middleware) tại Server: Hễ có request từ Client gửi tới, Server sẽ ghi nhận lại và xuất ra đúng chuẩn JSON đã thống nhất ở Chặng 0.
- 📦 **Điểm bàn giao 1 (Handover cho TV2):** TV1 đóng gói source code (hoặc đẩy lên Git/Docker), cung cấp danh sách API (POST /login, GET /resource) và tài liệu mô tả vị trí lỗ hổng cho TV2.

3.Chặng 2: Thành viên 2 xây dựng Kịch bản Tấn công & Bắt gói tin (Ngày 3 - Ngày 4):Đầu vào: App/Server của TV1 | Đầu ra: Bộ Script Hack + Luồng log tấn công.

Sau khi nhận hệ thống đang chạy từ TV1, **Thành viên 2** bắt đầu đóng vai Hacker xâm nhập vào hệ thống đó.

- **Bước 2.1:** Khảo sát hệ thống của TV1, thử nghiệm gửi request bình thường để kiểm tra phản hồi của Server.
- **Bước 2.2:** Viết kịch bản **Tấn công trên đường truyền (Network Sniffing / MitM)**: Thiết lập công cụ/script lắng nghe trên cổng giao tiếp giữa Client và Server của TV1, bắt lấy gói tin chứa username/password khi người dùng bấm đăng nhập.
- **Bước 2.3:** Viết kịch bản **Tấn công thẳng vào App/Server**:
    - Script tự động bơm mã độc (SQL Injection payload) vào API POST /login của TV1 để chiếm quyền truy cập tài nguyên mà không cần mật khẩu.
    - Script bắn request liên tục để thử mật khẩu hoặc gây nghẽn (Brute-force / DoS cơ bản).
- **Bước 2.4:** Gắn bộ phát tín hiệu vào các script tấn công: Mỗi khi Hacker bắt được gói tin trên đường truyền hoặc gửi mã độc đi, script sẽ tự động xuất ra một bản tin JSON báo cáo hành vi của Hacker.
- 📦 **Điểm bàn giao 2 (Handover cho TV3):** TV2 bàn giao toàn bộ hệ thống (gồm cả phần của TV1 + các script tấn công của TV2). Lúc này hệ thống đã có đủ 2 luồng dữ liệu JSON bắn ra từ cả phía App/Server lẫn phía Hacker.

4.Chặng 3: Thành viên 3 xây dựng Website Góc nhìn Thượng Đế (Ngày 5 - Ngày 6):Đầu vào: Luồng sự kiện từ TV1 & TV2 | Đầu ra: Website trực quan hóa toàn cảnh.

Thành viên 3 nhận toàn bộ "chiến trường" từ TV1 và TV2, gom tất cả dữ liệu về một mối để hiển thị lên màn hình quan sát.

- **Bước 3.1:** Dựng God-View Server (sử dụng WebSocket) làm trạm thu phát trung tâm, lắng nghe toàn bộ bản tin JSON do Server của TV1 và Script của TV2 bắn về.
- **Bước 3.2:** Thiết kế giao diện Website "Góc nhìn Thượng Đế" gồm 2 khu vực chính:
    - **Sơ đồ kiến trúc trực quan (Topology Map):** Hiển thị 3 thực thể Client ↔ Server và Hacker. Khi nhận được JSON sự kiện từ bước của TV1 và TV2, giao diện sẽ vẽ hiệu ứng luồng dữ liệu di chuyển tương ứng (màu xanh cho giao tiếp bình thường của TV1, màu đỏ cho đòn tấn công/nghe lén của TV2).
    - **Bảng đối chiếu sự kiện (Split-View Inspector):** Hiển thị song song theo thời gian thực: *Bên trái* là những gì Client/Server đang xử lý (ví dụ: câu lệnh SQL bị biến dạng), *Bên phải* là những gì Hacker đang thu thập được (ví dụ: mật khẩu bắt được trên đường truyền).
- **Bước 3.3:** Tạo bảng nút bấm điều khiển (Trigger Panel) ngay trên web để có thể bấm kích hoạt luồng chạy của TV1 và TV2 phục vụ cho việc demo báo cáo.

- 📦 **Điểm bàn giao 3 (Hoàn tất luồng Tuần 1):** TV3 tích hợp xong giao diện, chạy thử nghiệm từ đầu đến cuối.